

Security Vulnerability Detection in Python using Graph database

Visesh Bentula

Department of Computer Science
Texas A&M University-San Antonio
vbent01@jaguar.tamu.edu

Abstract—This work presents a lightweight graph-based vulnerability analysis system for Python programs using AST parsing [1] and Neo4j graph modeling [2]. Source code is transformed into a property graph of functions, variables, and CALL relationships, enabling global visibility of how execution flows reach dangerous operations. Cypher rules automatically detect six CWE categories—including hard-coded secrets (CWE-259), path traversal (CWE-22), command injection (CWE-78), unsafe `eval()` (CWE-94), insecure deserialization (CWE-502), and weak hashing (CWE-327)—identifying 16 vulnerable nodes within a 77-node program graph.

Compared with traditional scanners such as CodeQL [3], Joern [4], and Semgrep [5], this approach provides clear, interpretable visualizations of vulnerability propagation across functions rather than isolated warnings. Inspired by the Juliet Test Suite [6], which provides structured CWE examples for C/C++ but lacks Python coverage, this work adapts the idea for Python.

I. INTRODUCTION

Python applications frequently include insecure patterns such as hard-coded secrets, unsafe shell commands, insecure deserialization, weak hashing, and unchecked user input. Traditional static analyzers can flag individual issues but struggle to show how vulnerabilities propagate across functions toward dangerous sinks like `system()`, `eval()`, or `open()`.

Graph-based representations offer a richer view of program structure by modeling functions, variables, and call relationships as connected nodes and edges. However, tools such as CodeQL, Joern, and Semgrep provide limited Python support, shallow interprocedural reasoning, or require complex rule sets. Machine-learning approaches demand large labeled datasets, while benchmarks like the Juliet Test Suite lack Python coverage.

This work presents a lightweight static-analysis approach that extracts a Python program’s Abstract Syntax Tree (AST), builds a function-call graph, and loads it into Neo4j for interactive CWE-driven querying. Using Cypher rules, the system detects vulnerabilities such as hard-coded secrets, command injection, path traversal, insecure `eval`, weak hashing, and unsafe deserialization. Visualizing these flows in Neo4j exposes multi-step attack paths more clearly than linear code review.

II. BACKGROUND & RELATED WORK

This section reviews the foundations of static vulnerability detection, the motivation for graph-based representations of

code, and the major tools and datasets relevant to this work.

A. Static Analysis for Vulnerability Detection

Static analysis examines program structure without executing code, enabling the early detection of issues such as hard-coded secrets, unsafe API usage, shell command construction, insecure deserialization, and weak hashing. Python’s `ast` module provides a lightweight mechanism for extracting functions, calls, and variable assignments directly from source code, making it suitable for scalable analysis pipelines [1].

Traditional static analyzers offer strong scalability and low runtime cost, but they often operate linearly over files and struggle to identify how vulnerabilities propagate across interconnected functions. As a result, complex flows into dangerous sinks like `system()`, `eval()`, or `open()` may remain hidden from developers.

B. Graph-Based Representations of Code

Graph-based program models provide a richer structural view by representing functions, variables, and API calls as nodes, with edges capturing relationships such as `CALLS`, `defines`, or `uses`. This enables fine-grained reasoning about data and control flow, making it easier to trace multi-step vulnerability propagation and identify high-risk execution paths.

Neo4j, a widely used property-graph database, supports expressive querying through Cypher and naturally accommodates code graphs consisting of functions and call edges [2]. Security properties can be expressed as graph patterns (e.g., flows into dangerous APIs), allowing vulnerabilities to be detected using declarative rules rather than heavyweight analysis techniques.

C. Existing Tools and Approaches

CodeQL performs declarative, database-style analysis over large code bases and offers powerful semantic queries, but it requires complex custom rules and significant tuning for each project [3].

Joern builds a Code Property Graph (CPG) to support deep vulnerability search and data-flow reasoning but primarily targets C/C++ and provides limited support for Python, along with substantial setup overhead [4].

Semgrep offers lightweight, rule-based pattern matching with excellent speed and usability but cannot track interprocedural flows or reason about how data moves across multiple functions [5].

Graph neural network (GNN) approaches attempt to learn vulnerability patterns from code graphs, but they require large labeled datasets and expensive training pipelines, making them impractical for lightweight or explainable analysis.

D. Juliet Test Suite

The Juliet Test Suite (JTS) [6] is a widely used benchmark collection of synthetic vulnerable programs curated by NIST. It provides pairwise “good” and “bad” implementations for hundreds of CWE categories, making it valuable for validating static-analysis tools and machine-learning models. However, Juliet primarily targets C/C++ and Java; no official Python equivalent exists. This gap motivates constructing custom Python examples that mirror common CWE vulnerabilities, enabling evaluation of graph-based techniques in a Python context.

III. PROBLEM DEFINITION & RESEARCH QUESTIONS

Modern Python applications frequently contain security weaknesses such as unsanitized input, unsafe shell commands, insecure deserialization, weak hashing, and hard-coded secrets. Traditional static analyzers can detect these issues in isolation but struggle to reveal how vulnerabilities propagate across multiple functions toward dangerous sinks like `system()`, `eval()`, or `open()`. More advanced approaches (e.g., deep taint analysis or GNN-based models) require substantial computation and large training datasets, limiting their practicality for lightweight use.

This work examines whether representing Python programs as graphs can support more interpretable and accurate vulnerability detection by exposing the relationships among functions, variables, and high-risk API calls.

We investigate the following research questions:

RQ1: Can AST parsing automatically construct a structural function-call graph from Python code?

RQ2: Can Neo4j Cypher queries reliably detect vulnerability patterns corresponding to common CWE categories?

RQ3: Does graph visualization help developers trace and understand vulnerable flows across functions?

RQ4: Can this graph-based method serve as a lightweight alternative to heavier static-analysis frameworks for Python?

IV. APPROACH OVERVIEW

Our system performs lightweight static analysis by extracting the Abstract Syntax Tree (AST) of a Python program, identifying functions, variables, and `CALLS` relationships, and loading the resulting graph into Neo4j for interactive querying. Vulnerability rules are encoded as Cypher patterns that detect flows into dangerous sinks such as `system()`, `eval()`, `pickle.loads()`, and unsafe `open()` usage. Each match is labeled with the corresponding CWE category (e.g., CWE-259, CWE-22, CWE-78, CWE-94, CWE-502, CWE-327), producing a color-coded vulnerability map.

V. IMPLEMENTATION DETAILS

The implementation consists of a Python AST parser, a Neo4j upload module, and a rule-based detection engine. The parser walks the AST to extract function definitions, function calls, and string constants. These are converted into Neo4j nodes and edges. Cypher queries then tag vulnerable nodes based on known patterns (e.g., command injection via `system()`, path traversal via `open()`, weak hashing through `md5()`). The entire workflow runs in a few seconds on a single script and requires no heavyweight analysis tools.

VI. EXPERIMENTAL EVALUATION

We evaluated the system on a deliberately vulnerable Python sample (200 lines) containing common CWE patterns. Neo4j produced a program graph with 77 nodes and 76 call edges. A total of 16 vulnerable nodes were identified across six CWE categories. The graph view made multi-step attack chains (e.g., `main_demo` → `list_directory` → `system`) immediately visible, demonstrating the advantage of graph-based reasoning over linear static analysis.

VII. CASE STUDIES

We highlight three representative findings: (i) hard-coded secrets (CWE-259) feeding authentication logic, (ii) unsafe `eval()` execution (CWE-94), and (iii) command injection paths reaching `system()` and `Popen()` (CWE-78). In each case, Neo4j exposes both the vulnerable endpoint and the call chain leading to it, enabling clearer understanding of how insecure flows arise.

VIII. CONCLUSION

This work demonstrates that representing Python code as a graph enables effective, interpretable vulnerability detection using only AST analysis and Cypher rules. The system identifies multiple CWE categories, visualizes interprocedural flows, and reveals attack paths that conventional line-by-line scanners often miss. Future extensions include multi-file analysis, automatic report generation, and ML-assisted risk scoring.

REFERENCES

- [1] *Python ast module documentation*, <https://docs.python.org/3/library/ast.html>, Accessed: 2025-07-20.
- [2] *Neo4j graph database*, <https://neo4j.com/developer/>, Accessed: 2025-07-20.
- [3] GitHub Security Lab, “Codeql: Declarative static analysis for large-scale codebases,” in *GitHub Engineering Paper Series*, 2022. [Online]. Available: <https://codeql.github.com/>.
- [4] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Joern: A graph-based code analysis platform,” in *Proc. IEEE Security and Privacy Workshops*, 2016.
- [5] *Semgrep: Lightweight static analysis for modern languages*, <https://semgrep.dev/>, Accessed: 2025-07-20.
- [6] *Juliet test suite*, <https://samate.nist.gov/SRD/>, A large benchmark of synthetic CWE vulnerabilities. Accessed: 2025-07-20.